# A Comparative Algorithmic Analysis of Exact Search Methods for Urban Route Optimization: Re-evaluating the 17-City Traveling Salesman Problem

Alexandros Panagiotakopoulos
*Department of Electrical and Computer Engineering, Hellenic Mediterranean University, Heraklion, Crete, Greece*

***Abstract*—**The Traveling Salesman Problem (TSP) remains a cornerstone of NP-hard combinatorial optimization, with significant implications for urban logistics and resource management. This study presents a rigorous comparative implementation of two exact algorithmic approaches—Depth-First Search (DFS) with branch-and-bound pruning and a systematic Breadth-First Search (BFS)—to solve a realistic 17-city urban routing instance in Bolgatanga, Ghana. While previous literature concerning this specific instance reported an optimal route of 5080.74 m, the dual-language validation performed in this research (implemented in C# and Ruby) consistently identifies a superior optimal Hamiltonian cycle of 4497.55 m. This represents a significant reduction of 583.19 m (approximately 11.5%) compared to prior published results. The implementations incorporate robust data-validation mechanisms to address asymmetric distance relationships and numerical inconsistencies within the original dataset. The findings demonstrate that while DFS offers superior memory efficiency through recursive call stacks, both exact methods provide identical optimality guarantees for problems of this scale. This research provides a necessary correction to the existing TSP literature for this region and establishes a high-fidelity baseline for future heuristic and metaheuristic comparisons in urban delivery optimization.

***Index Terms*—**Traveling salesman problem, combinatorial optimization, branch and bound, breadth-first search, depth-first search, exact algorithms, urban route optimization.

## I. INTRODUCTION

The Traveling Salesman Problem (TSP) is a fundamental NP-complete combinatorial optimization problem, with a well-documented history spanning decades of algorithmic inquiry [5], and critical applications in logistics, manufacturing, and network routing. The core objective of the TSP is to identify the shortest possible Hamiltonian cycle in a weighted graph, where vertices represent discrete locations and edges represent inter-location distances or costs. Given that the number of possible routing permutations grows factorially (n!), brute-force enumeration becomes computationally intractable for instances exceeding 20–25 cities.

This exponential complexity has historically driven the academic community toward heuristics and metaheuristics, such as genetic algorithms [3], simulated annealing, and ant colony optimization. However, while heuristics offer rapid computational approximations, they inherently sacrifice the guarantee of absolute mathematical optimality — a distinction long emphasized in surveys contrasting exact and approximate algorithmic families [7]. While modern exact solvers are capable of certifying optimality for instances vastly larger than the one considered here [2], problems constrained to micro-logistics scale ($n \leq 20$) carry no comparable excuse for an inexact result: they must be exhaustively solved to establish a definitive ground truth. The failure to identify the absolute mathematical optimum in the prior literature on this exact instance [1] illustrates the operational risk of applying heuristic or insufficiently bounded exact methods even when the problem size falls comfortably within tractable limits.

This paper provides a rigorous comparative analysis between exact depth-first search (DFS) methods and systematic breadth-first search (BFS) approaches for solving a specific 17-city TSP instance. By examining the performance trade-offs, space complexity, and computational efficiency of these algorithms, this study highlights the viability of exhaustive enumeration techniques. Crucially, this research serves as a formal rebuttal to the findings published in [1] regarding this specific routing instance, demonstrating the absolute necessity of rigorous validation in applied computational optimization.

## II. RELATED WORK AND PRIOR TSP RESEARCH

Recent literature has extensively explored the application of TSP optimization in local urban micro-logistics. Notably, Najat and Boah [1] applied TSP optimization to a 17-city routing problem for a street vendor operating in Bolgatanga, Ghana. Their study employed a Branch and Bound methodology as implemented in Version 3 of a third-party TSP Solver and Generator utility [6], reporting an optimal route of 5080.74 m and contrasting it favorably against the vendor's intuitive daily routes (which varied between 5194.83 m and 5470.33 m). While their work successfully highlighted the practical utility of operations research in small-scale commerce, the present study independently reconstructs and evaluates this specific instance, applying rigorous dual-paradigm programmatic validation to verify the absolute structural integrity of the proposed routes.

In computational optimization, traditional Branch and Bound algorithms perform depth-first exploration combined with strategic pruning, maintaining a linear O(n) space complexity — a family of formulations and solution procedures that has been broadly surveyed in the literature [8]. Conversely, breadth-first approaches explore the search space horizontally using queue-based structures, demanding $O(b^d)$ space. This research contrasts these two paradigms not only in theoretical complexity but in practical application, utilizing strongly typed compiled structures (C#) and dynamically typed interpreted structures (Ruby) to ensure environment-agnostic result verification.

## III. METHODOLOGY

### *A. Problem Representation and Data Validation*

TSP instances are mathematically represented as weighted graphs G = (V, E) with a distance matrix D, where D[i][j] represents the inter-city spatial distance. As foundational combinatorial optimization literature establishes, a standard symmetric TSP formulation requires strict bidirectional equality such that D[i][j] = D[j][i] for every node pair [4], [9]. A critical pre-processing phase introduced in this study therefore performs comprehensive bidirectional validation of the dataset against this requirement.

A thorough inspection of the dataset published in the original study revealed explicit numerical inconsistencies that violate this principle outright. The matrix contains conflicting mirrored entries: the distance from node 1 to node 12 is recorded as $d_{1,12}$ = 627.43, while the reverse path is recorded as $d_{12,1}$ = 672.43. This is not a rounding artifact — it is a structural defect that silently converts the problem into an Asymmetric TSP without acknowledgment or appropriate algorithmic handling in the original work [1]. To address this anomaly, both implementations parse the dataset to detect isolated nodes, asymmetric paths, and connectivity violations before executing the core algorithms, ensuring no corrupted edge weight propagates into the final route computation.

### *B. Depth-First Search with Branch and Bound (C#)*

The exact DFS algorithm, developed in C#, employs recursive backtracking to systematically enumerate all tour permutations. The implementation integrates implicit branch-and-bound pruning mechanics by maintaining a global bounds variable. When a partial route's accumulated distance exceeds the current established upper bound, the traversal tree is immediately pruned, significantly reducing the search space.

```
// Core pruning logic excerpt
if (currentDistance >= bestDist) return;

if (currentRoute.Count == totalNodes) {
    if (dist.ContainsKey((curr, start))) {
        currentDistance += dist[(curr, start)];
        if (currentDistance < bestDist) {
            bestDist = currentDistance;
            bestRoute = new
List<string>(currentRoute);
        }
    }
}
```

### *C. Breadth-First Search (Ruby)*

To ensure cross-validation, a secondary systematic algorithm was developed using Ruby. This approach employs queue-based, level-by-level exploration. The algorithm maintains state tracking via node-path-distance triplets stored within a FIFO queue. Unlike depth-first traversal, BFS examines all concurrent paths at a specific depth tier before progressing deeper. While this approach incurs higher memory overhead ($O(b^d)$), it provides an exhaustively distinct exploration ordering, guaranteeing that the final output is independent of procedural biases inherent to a single algorithm's traversal path.

## IV. EXPERIMENTAL RESULTS AND REBUTTAL

A central objective of this research is the direct computational evaluation of the prior solution proposed for the 17-city urban routing instance in Bolgatanga. The original study by Najat and Boah [1] concluded with an optimal route distance of 5080.74 m.

However, the rigorous dual-algorithm computational analysis performed in the present study reveals a significant mathematical discrepancy. Both the C# DFS approach and the Ruby BFS approach—operating completely independently—exhaustively evaluated the problem space and consistently identified a significantly shorter optimal Hamiltonian cycle with a total distance of **4497.55 m.**

### *A. The Verified Optimal Route*

The true optimal route verified by both exact methods is sequenced as follows:

*Fruits Seller's House → Carpentry Shop → Post Office → Jubilee Park → Straw Market → Ghana Commercial Bank → National Investment Bank → Old Market → Kotokoli Line → Transport Station → New Market → Cola Market → Abattoir → Goil Filling Station → Ayia Street → Taxi Rank → OA (MTN Office) → Return to Origin.*

**TABLE I**
**COMPARISON OF REPORTED OPTIMAL ROUTE DISTANCES**

| Study / Method | Reported Distance | Status |
|---|---|---|
| Najat & Boah (B&B) | 5080.74 m | Suboptimal [1] |
| **Current Study (DFS & BFS)** | **4497.55 m** | **Optimal (Verified)** |

This newly discovered route reduces the total traversed distance by **583.19 m** compared to the originally published conclusion. The absolute consistency of this result across two fundamentally different programmatic exploration strategies provides robust, undeniable validation of its optimality. This substantial 11.5% variance is attributable to incomplete search-space exploration on the part of the original authors [1], compounded by the explicit numerical errors within the distance matrix their analysis relied upon.

## V. CONCLUSION

This study successfully implemented and cross-validated two exact search algorithms to solve a 17-city urban routing TSP instance. By identifying a mathematically optimal route of 4497.55 m, this research provides a definitive correction to Najat and Boah [1], who erroneously reported an optimal distance of 5080.74 m. The findings underscore a fundamental principle in operations research: instances that fall within the threshold of computational tractability ($n \leq 20$) must be evaluated using robust, exact methodologies to prevent the formalization of suboptimal baselines. Future work will explore the integration of dynamic constraints, such as real-time traffic data, onto this verified optimal base layer.

## REFERENCES

[1] A. S. Najat and D. K. Boah, "Analysis of the Operating Characteristics of Fruits' Seller in Bolgatanga as a Travelling Salesperson Problem," American Journal of Computational and Applied Mathematics, vol. 11, no. 4, pp. 71–77, 2021.

[2] D. L. Applegate, R. E. Bixby, V. Chvatal, and W. J. Cook, The Traveling Salesman Problem: A Computational Study. Princeton, NJ, USA: Princeton University Press, 2006.

[3] J. H. Holland, Adaptation in Natural and Artificial Systems. Cambridge, MA, USA: MIT Press, 1992.

[4] E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy Kan, and D. B. Shmoys, The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization. Chichester, UK: Wiley, 1985.

[5] A. Schrijver, "On the history of combinatorial optimization (till 1960)," in Handbook of Discrete Optimization, Amsterdam, The Netherlands: Elsevier, 2005, pp. 1–68.

[6] O. L. Serdiuk, Travelling Salesman Problem Solver and Generator, Version 3, 2007.

[7] G. Laporte, "The traveling salesman problem: An overview of exact and approximate algorithms," European Journal of Operational Research, vol. 59, pp. 231–247, 1992.

[8] T. Bektas, "The multiple traveling salesman problem: an overview of formulations and solution procedures," Omega, vol. 34, pp. 209–219, 2006.

[9] B. Korte, "The Traveling Salesman Problem," in Combinatorial Optimization, Algorithms and Combinatorics, vol. 21, Berlin, Germany: Springer, 2008.